Null, Constructors, and Shared State

CSCI 2113 — Lab 3: when a reference points at nothing

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

September 12, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

Outline

1. Where Lab 3 Fits
2. Null and NullPointerException
3. Constructors: Overloading and Chaining
4. Composition and Shared State
5. The Assignment: Rides and the Log
6. Wrap-up
7. Acknowledgement

ng Aiersilan | CSCI 2113

Where Lab 3 Fits

ng Aiersilan | CSCI 2113 | F

You already have a dispatch board

Lab 2 left you with **Driver** and **DispatchBoard**, and a board that could already answer *who is free* and *who is nearest*.

- Every **Driver** was built the same way — **four** arguments, every time.
- Every field held a real value. Nothing was ever missing.

Real dispatch is not that tidy. A driver can be on the board with no vehicle assigned yet.

Today's question

What does Java do when a reference points at **nothing** — and how do you stop that from ending your program?

Four ideas, one dispatch board

1. **null**, and the crash it causes — **NullPointerException**.
2. **Constructor overloading**, chained with `this(...)`.
3. A **static** field, shared by the whole class.
4. **Composition** — one object *has-a* another.

All four meet in one place: a **Driver** that has a **Vehicle** — or has none.

ng Aiersilan | CSCI 2113

Null and NullPointerException

ng Aiersilan | CSCI 2113 | F

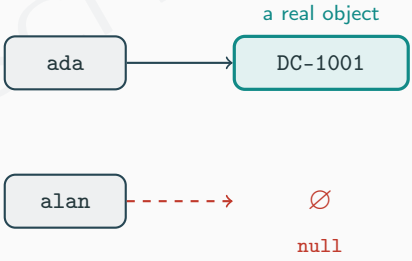
A reference that points at nothing

A variable of a class type never *holds* an object. It holds a **reference** — an arrow to one.

- `new Vehicle(...)` builds the object and hands you the arrow.
- **null** is the arrow that points nowhere.

`null` is a perfectly legal value. You may store it, pass it, return it, compare it, even print it.

The man who put it in a language in **1965** later called it his "billion-dollar mistake" [3].



Step 1: predict, then run

```
public class DispatchBoard {
    public static void main(String[] args) {
        Driver d = null;
        System.out.println(d == null);   // ???
        System.out.println(d);           // ???
        System.out.println(d.getName()); // ???
    }
}
```

Write all three answers down *before* you compile. ⚠ One of these lines never prints anything at all.

ng Aiersilan | CSCI 2113

Two print. One stops the program.

```
true
null
Exception in thread "main" java.lang.NullPointerException: ...
```

- `d == null` — comparing a reference never touches an object: `true`.
- `println(d)` — prints the four letters `null`, and carries on.
- `d.getName()` — ✖ this one asks the object a question. There is no object.

Nothing after the crash runs. The program stops on that line.

ng Aiersilan | CSCI 2113

The dot is the dividing line

Everything you do to the **reference itself** is safe: assign it, pass it, return it, compare it with ==.

`println` is safe for a quieter reason — it hands the reference to `String.valueOf`, which checks for null first and returns the text `"null"`.

Exactly one move needs a real object on the far end: following the arrow — to call a method, read a field, index an array, or unbox a number. We call it **dereferencing**, and the dot is how you usually write it.

Never throws

```
d == null
println(d)
"x " + d
```

Dereferences

```
d.getName()
d.getZone()
rides[i]
```

Read the whole message — it names the culprit

```
Exception in thread "main" java.lang.NullPointerException:
        Cannot invoke "Driver.getName()" because "d" is null
        at DispatchBoard.main(DispatchBoard.java:6)
```

- **What went wrong** — **NullPointerException**.
- **What you tried** — `Cannot invoke "Driver.getName()"`.
- **Which reference was empty** — `because "d" is null`.
- **Where** — `DispatchBoard.java:6`, the exact line.

It is one console line, wrapped here to fit. That `because` clause arrived in JDK **14** and prints by default from JDK **15** [4]. The *variable name* `"d"` needs debug information — your IDE compiles that way; a bare `javac` keeps the line number but prints `"<local1>"` for the name.

Step 4: check before you follow the arrow

```
Vehicle v = alan.getVehicle();
if (v != null) {
    System.out.println(alan.getName() + " drives " + v.getPlate());
} else {
    System.out.println(alan.getName() + " has no vehicle");
}
```

```
Alan has no vehicle
```

The program now ends normally. The shape is worth memorising:
name the reference, test it, then use it inside the branch that knows it is real.

ng Aiersion | CSCI 2113

Constructors: Overloading and Chaining

ng Aiersilan | CSCI 2113 | F

One name, several ways in

A class may declare many constructors, as long as their **signatures** differ — that is, the *number and types* of the parameters [2].

```
new Vehicle("DC-1002", 6)   // two arguments -> Vehicle(String, int)
new Vehicle("DC-1001")      // one argument  -> Vehicle(String)
```

The compiler picks which one to run, at compile time, from the arguments you wrote. Parameter *names* are not part of the signature, so renaming them does not create a second constructor:

```
error: constructor Vehicle(String,int) is already defined in class Vehicle
```

ng Aiersilan | ESCI 2113

Step 2: two doors, one rule

```
public class Vehicle {
    private String plate;
    private int seats;

    public Vehicle(String plate, int seats) {
        this.plate = plate;
        if (seats < 1) { this.seats = 1; }     // the rule lives here
        else           { this.seats = seats; }
    }

    public Vehicle(String plate) {
        this(plate, 4);                        // ... and only here
    }
}
```

The one-argument constructor assigns *nothing*. It forwards — so the seat rule is written once and obeyed twice.

Ersilan Aydin | CSCI 2113

What comes out

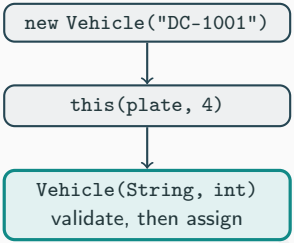
```
new Vehicle("DC-1001")
new Vehicle("DC-1002", 6)
new Vehicle("DC-1003", 0)
```

```
DC-1001 (4 seater)
DC-1002 (6 seater)
DC-1003 (1 seater)
```

- The default of **4** seats is written once, in the short constructor.
- The floor of **1** seat is written once, in the long one — and the short one still gets it, because it goes through.

Duplicate the rule in both constructors and the two copies will drift apart. That is the bug this pattern prevents.

`this(...)` — delegate, do not duplicate



Two rules the compiler enforces:

- `this(...)` must be the first statement in the constructor [2].
- A constructor may not chain to itself.

Break either one and you get:

```
error: call to this must be first
       statement in constructor
error: recursive constructor invocation
```

⚠ On Java **25** that first message reads differently, and the rule itself is looser — see the backup slide.

ng Aiersilan | CSCI 2113

Composition and Shared State

ng Aiersilan | CSCI 2113 | F

Step 3: a Driver *has a* Vehicle

```
public Driver(String name, int zone,
              boolean available, double rating) {
    this(name, zone, available, rating, null);
}

public Driver(String name, int zone,
              boolean available, double rating,
              Vehicle vehicle) {
    ...
    this.vehicle = vehicle;
}
```

A field whose type is a class is just another **reference**. Same rules, same risks.

The four-argument constructor supplies `null` on your behalf — so "no vehicle" is a real, ordinary state, not an accident.

ng Aiersilan | CSCI 2113

Four predictions, one crash

```
Driver ada = new Driver("Ada", 3, true, 4.8, new Vehicle("DC-1001"));
System.out.println(ada);
System.out.println(ada.getVehicle().getSeats());
Driver alan = new Driver("Alan", 2, true, 4.0);
System.out.println(alan);
System.out.println(alan.getVehicle().getSeats());
```

```
Ada     | zone 3 | 4.8 | AVAILABLE   | DC-1001 (4 seater)
4
Alan    | zone 2 | 4.0 | AVAILABLE   | null
Exception in thread "main" java.lang.NullPointerException:
    Cannot invoke "Vehicle.getSeats()" because the return value of
    "Driver.getVehicle()" is null
```

The driver text is your Lab 2 `toString()`; everything after the last `|` is what Lab 3 added. The exception is one console line, wrapped here to fit.

ng Aiersilan | CSCI 2113

Same `null`, two very different fates

```
" | " + this.vehicle
```

String concatenation converts a null reference to the text `"null"` [2]. Nothing is asked of the object. ✔ Prints fine.

```
alan.getVehicle().getSeats()
```

Two dereferences on one line. The first succeeds; the second has no object. ✖ Throws.

💡 The message tells you *which* one failed: "the return value of `Driver.getVehicle()` is null" — not `alan`.

Step 5: whose fact is it?

```
private static int nextId = 1;
private int id;

Driver(...) {
    ...
    this.id = nextId;
    nextId++;   // no this!
}
```

Ask one question of every field:
whose fact is this? Is it true of one object, or of the class as a whole?

- A driver's own `id` — about *that* driver. An **instance field**.
- The *next* id to hand out — about no driver in particular. A **static field**, stored once, for the whole class.

`nextId` needs no `this`, because it does not belong to `this`.

ng Aiersilan | CSC 2113

Where you put those two lines decides everything

Where the two `id` lines live	`ada.getId()`	`alan.getId()`
five-argument constructor only *(as specified)*	**1**	**2**
both constructors	**1**	**3**
`nextId` declared without `static`	**1**	**1**

- `alan` is built with **four** arguments, which never mention `nextId` — yet he still gets **2**, because that constructor chains through the five-argument one.
- Put the lines in both, and `alan` is numbered twice: he ends up **3**, and **2** is silently lost.
- Drop `static`, and every driver carries a private counter that starts at **1**. Every id is **1**.

The Assignment: Rides and the Log

ng Aiersilan | CSCI 2113 | F

Three new classes, three different jobs

Ride

An **information holder**. Knows a driver, a rider, a fare, an id — and answers questions about them.

RideLog

A **structurer**. Holds many rides and answers questions *across* them.

Dispatch

A **controller**. Decides what happens, in what order, and tells the others to do it.

Those three names are a standard design vocabulary [6]. Naming a class's job early keeps its methods honest.

Part 1: put the vehicles on the board

```
static Driver[] drivers = {
    new Driver("Ada",     3, true,  4.8, new Vehicle("DC-1001", 4)),
    new Driver("Grace",   1, false, 5.0, new Vehicle("DC-1002", 6)),
    ...
    new Driver("Alan",    4, true,  4.4, new Vehicle("DC-1006", 7))
};
```

```
Ada     | zone 3 | 4.8 | AVAILABLE   | DC-1001 (4 seater)
Barbara | zone 2 | 4.9 | UNAVAILABLE | DC-1005 (4 seater)
```

Pad the name to **7** and the status to **11** so every column lines up.

Then `nearestAvailableWithSeats(riderZone, partySize)`: your Lab 2 `nearestAvailable`, with one extra filter — skip any driver whose vehicle has fewer than `partySize` seats. Same return type, same tie-breaks: closest wins, then higher rating, then array order.

ng Aiersilan / CSCI 2113

Part 2: Ride

Four private fields: `driver`, `riderName`, `fare`, `id`.

The id is not a constructor parameter. It comes from a **static** counter — exactly the pattern you just built in **Driver**.

Getters for all four.

```
Ride #1: Ana with Ada | $20.00
```

That exact shape is the target for `toString()`. Use `String.format("%.2f", fare)` so **20.0** prints as **20.00** and **13.75** keeps both digits.

ng Aiersilan | CSCI 2113

Part 3: RideLog

```
private Ride[] rides = new Ride[100];
private int count = 0;
```

- `addRideRecord(ride)` — store at index `count`, then increase `count`. The array has **100** slots; `count` says how many are real.
- `count()` — how many rides are recorded. A method named like the field it reports on; Java keeps the two apart.
- `findMostExpensiveRide()` — walk the first `count` entries, keep the largest fare. ⚠ With nothing recorded, return `null` — and now you know what the caller must do with it.
- `findRide(driver)` — return the first ride that driver served, or `null`.

`findRide`: reuse the `equals` you already wrote

```
if (rides[i].getDriver().equals(driver)) { return rides[i]; }
```

The assignment is explicit: use the `equals` from Lab 2 — same name and zone — not ==.

== asks	`.equals` asks
Are these the same object?	*Do these count as the same driver?*
Identity.	Equivalence, on *your* terms.

Here both happen to answer `true`, because **Dispatch** hands the **Ride** the very **Driver** it pulled from the array. But only `.equals` still works when the caller looks the driver up some other way — and defining what "the same" means is the class's job, not =='s. ⚠ Guard the argument: `driver` may be `null`.

Part 4: `requestRide`, in order

```
public static Ride requestRide(String riderName, int riderZone,
                               int destZone, int partySize) { ... }
```

1. Find a driver with `nearestAvailableWithSeats`. ⚠ If there is none, `return null immediately` — change nothing else.
2. Fare: `BASE_FARE + FARE_RATE * distance[riderZone][destZone]` — **2.50** base plus **1.25** per unit of distance.
3. Build the **Ride**, record it in the log, mark the driver unavailable, return it.

Step **1** returning early is the whole point: a failed request must leave the board exactly as it found it.

ng Aiersilan | CSCI 2113

Trace the four requests

Request	Party	Still eligible	Driver	Result
Ana, zone 0→4	3	Ada, Linus, Alan	Ada	`Ride #1` — **$20.00**
Ravi, zone 2→0	6	Alan only	Alan	`Ride #2` — **$13.75**
Sam, zone 1→3	8	nobody	—	`null`
Lee, zone 3→3	1	Linus, Ken	Linus	`Ride #3` — **$2.50**

- Ravi is capacity alone: only Grace and Alan seat **6**+, and Grace is off duty. Ada's booking is beside the point — her car seats **4**.
- Lee is where availability decides. Every car seats **1**+, so Ada (`distance[3][3]` is **0**) would have won — but she and Alan are out on rides, so Linus (**5** min) beats Ken (**8**).
- Lee is `Ride #3`, not #4: Sam built no **Ride**, so he consumed no id. Lee stays inside zone **3**, so his fare is the base alone.
- **$20.00** pins the arithmetic: `2.50 + 1.25 * 14`. If your fare differs, check your zone matrix and your indices.

Wrap-up

ng Aiersilan | CSCI 2113 | F

Habits, and the traps they avoid

Do

- ✔ Name the reference, test it, then use it.
- ✔ Write each rule in one constructor; chain to it.
- ✔ Read the `because` clause, not just the line number.

Watch out

- ✖ `d.getName() != null` — already dereferenced.
- ✖ Copying the validation into both constructors.
- ✖ Forgetting `static`, so every id is **1**.

💡 Every trap above is the same mistake: assuming something is there.

Takeaways

1. ★ **null** is a legal value, not an error. Only dereferencing it is.
2. ★ `this(...)` lets a rule live in **one** place and still apply everywhere.
3. ★ **static** marks a fact about the *class*. Everything else belongs to an object.

In one sentence

Composition means a **Driver**'s **Vehicle** might be missing — and today was about saying so honestly instead of crashing.

Acknowledgement

ng Aiersilan | CSCI 2113 | F

Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

References

[1] J. Bloch. ***Effective Java.*** Addison-Wesley Professional, 3rd edition, 2018. ISBN 978-0-13-468599-1.

[2] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman. ***The Java Language Specification, Java SE 21 Edition.*** Oracle America, Inc., 2023. https://docs.oracle.com/javase/specs/jls/se21/html/.

[3] T. Hoare. **Null References: The Billion Dollar Mistake.** Presentation, QCon London 2009, 2009. https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/.

[4] G. Lindenmaier and R. Schmelter. **JEP 358: Helpful NullPointerExceptions.** OpenJDK JDK Enhancement Proposal, delivered in JDK 14, 2019. Code details are printed by default from JDK 15 onward. https://openjdk.org/jeps/358.

[5] Oracle. **Flexible Constructor Bodies.** Java SE 25 Language Updates, 2025. https://docs.oracle.com/en/java/javase/25/language/flexible-constructor-bodies.html.

[6] R. Wirfs-Brock and A. McKean. ***Object Design: Roles, Responsibilities, and Collaborations.*** Addison-Wesley Professional, 2002. ISBN 0-201-37943-0. Source of the role stereotypes (information holder, structurer, controller).

Thank you!

Questions?

Backup: `equals(Vehicle)` is an *overload*

```
Vehicle a = new Vehicle("DC-1001", 4);
Vehicle b = new Vehicle("DC-1001", 6);
System.out.println(a.equals(b));     // true  -> your equals(Vehicle)
Object o = b;
System.out.println(a.equals(o));     // false -> Object.equals(Object)
```

The lab's `equals(Vehicle other)` takes a **Vehicle**, so it *adds* a method rather than replacing `Object`'s. Which one runs is decided by the declared type of the argument, at compile time.

⚠ *Overriding* properly means writing `equals(Object)` and checking the type inside. That is beyond this lab — but it is why the distinction matters [1].

ng Aiersiani / CSCI 2113

Backup: why your message may name <local1>

```
compiled by an IDE (javac -g):
  ... because "d" is null
      at DispatchBoard.main(DispatchBoard.java:6)

compiled with javac -g:none:
  ... because "<local1>" is null
      at DispatchBoard.main(Unknown Source)

a bare javac (-g:lines,source):
  ... because "<local1>" is null
      at DispatchBoard.main(DispatchBoard.java:6)
```

Local variable *names* are optional debug information [4]. The rest of the message is rebuilt from the bytecode, so a clause like "the return value of `Driver.getVehicle()` is null" survives either way.

ng Aiersman / CSCI 2113

Backup: “first statement” in newer Java

In standard Java through **24**, `this(...)` and `super(...)` had to be the literal first statement [2] — the rule this lab assumes. (A preview of the relaxed form shipped from Java **22**, so “**24**” means standard, non-preview Java.)

Java **25** made **flexible constructor bodies** standard [5]: statements may now come *before* the call, as long as they never read the object under construction.

Allowed from 25

```
this.buildCount = 99;  // write
this(plate, 4);
```

Still rejected

```
System.out.println(this.seats);
this(plate, 4);        // read!
```

The reason never changed: no one may *observe* a half-built object.